Embedded Template Kullanım Kılavuzu

aifactory ile FPGA + firmware projelerinde günlük çalışma akışı: requirement, fazlar, Vivado/Vitis build, twin, board ve CI.

aifactory v0.71 (RQ-0044…RQ-0071) · Vivado/Vitis 2025.2 · ZedBoard referans kartı · 20 Ekim 2026

1. Template ne yapar

Embedded template, bir FPGA tasarımı ve üzerinde koşan C firmware'ini tek bir projede, requirement odaklı olarak geliştirmek içindir. aifactory projeyi üretir ve requirement akışını yürütür; Vivado ve Vitis işin yapıldığı araçlardır ve projenin kendi script'i (`scripts/flow.mjs`) ile uzaktaki build host'ta koşar. Kaynaklar metindir (HDL, XDC, Tcl, testbench, C); Vivado/Vitis proje dosyaları çıktıdır ve repoya girmez.

Her requirement bir dizi **faz**dan geçer; her fazı bir **gate** kapatır. Gate geçmeden requirement tamamlanmaz. Fazlar sırayla: `spec` → `sim` → `build` → `twin` → `board`.

spec	**sim**	**build**	**twin**	**board**
`designCheck` board dosyası, project.tcl ve block design tutarlı; mutlak yol yok	`hdlSim` her `tb/tb_*` xsim'de TEST PASSED basar	`fpgaBuild` · `firmwareBuild` synth/impl/bit/xsa, WNS/WHS ≥ 0, DRC 0; Vitis ELF üretilir	`twinTest` ELF hardware twin image'ında koşar, test.yaml kontrol eder	`boardTest` bitstream + ELF gerçek kartta JTAG ile; konsol beklenen satırı basar

Varsayılan fazlar `spec, sim, build`'dir. `twin` ve `board` requirement bazında seçilir; board gerçek kartı meşgul ettiği için sadece gerektiğinde, gece koşusunda ya da elle tetiklenir.

2. Proje yaratma

```
cd agentic/aifactory
pnpm factory -- new <proje> --template embedded --board zedboard --dir ..
# coding standard ile (bkz. §9):
pnpm factory -- new <proje> --template embedded --board zedboard --standard cstd --groups
misra,bytecraft --dir ..
```

`--board` lab envanterindeki (`aifactory/lab/inventory.json`) bir birimi seçer; Vivado host, lab host, hw_server, bot adresi, JTAG kablo seri numarası ve konsol portu `.env`'e otomatik dolar. Boş kalan ve sizin doldurmanız gerekenler: `GITLAB_TOKEN`, `BOT_API_TOKEN`, `TWIN_GITLAB_TOKEN` (gizli değerler envanterde tutulmaz). `pnpm factory -- env-check` eksikleri listeler.

> **Lab host'larına erişim: SSH ya da bot.** İki host var; her biri için `.env`'deki `*_ACCESS` ayarı (`ssh` | `bot`) yolu seçer. Developer'lar `bot` kullanır: SSH ve anahtar gerekmez, sadece `BOT_API_TOKEN`. `ssh` lab yöneticisi ve CI içindir. (Board için `bot` RQ-0055, twin için `TWIN_ACCESS=bot` RQ-0056 ile geldi; builder için `VIVADO_ACCESS=bot` RQ-0057 ile geldi.)
>
> - **192.168.1.3 (builder, Windows):** sim, build, sw. `VIVADO_ACCESS=bot`: bot'un builder'daki Windows agent'ı (`VIVADO_BOT_AGENT`) Vivado/Vitis'i koşturur; kaynak ve sonuçlar bot üzerinden gider gelir. `VIVADO_ACCESS=ssh`: kullanıcı `VIVADO_REMOTE_USER`, yani sizin builder'daki hesabınız. Hesabınız Administrators grubundaysa Windows OpenSSH anahtarı `~/.ssh/authorized_keys`'ten değil `C:\ProgramData\ssh\administrators_authorized_keys`'ten okur; `ssh-copy-id` Windows'a yazamaz. Parolayla bir kez:
>   `cat ~/.ssh/id_ed25519.pub | ssh <siz>@192.168.1.3 'powershell -NoProfile -Command "Add-Content -Path C:\ProgramData\ssh\administrators_authorized_keys -Value ([Console]::In.ReadToEnd().Trim())"'`
> - **192.168.1.2 (lab, Linux):** sadece board. `BOARD_ACCESS=bot`: güç bot'tan, konsol bot'un portmap köprüsünden (`BOARD_CONSOLE=lab/COM21`; aşama `pm start lab com21 owner=<proje>/<siz>` ile TCP akışı açar, bitince kapatır; konsol başkasındaysa durur ve kimde olduğunu söyler), JTAG builder'daki xsdb'den hw_server'a. `BOARD_ACCESS=ssh`: kullanıcı `root`; bu anahtar yalnızca lab yöneticisinde ve CI'da olur, developer'a verilmez.
>
> check aşaması 1.2'ye girmez; twin `TWIN_ACCESS=bot` ile lab'ın Docker'ında bot üzerinden koşar (ELF bot'un deposuna gider, sonuç arşiv olarak gelir), `ssh` ile kendi Docker'ınızda. `pnpm factory -- env-check` her aşamanın hangi yoldan hangi host'a gittiğini listeler. CI kendi anahtarlarını kullanır, sizinkilere ihtiyacı yoktur.

Proje GitLab'da açılır (`agentic` grubu), runner'ı Settings → CI/CD → Runners'tan etkinleştirin. Lab'ın ortak ayarları (Vivado host, lab host, SSH anahtarları, bot ve twin token'ları) GitLab instance düzeyinde tanımlıdır; projeye yalnızca karta özgü değerler girer: `pnpm factory -- ci-variables --write --with-secrets` bunu yapar ve ortak olanları atlar.

3. Proje yerleşimi

YOL	NE
`designs/<tasarım>/project.tcl`	Tasarımın tanımı: part, mode (`non_project` saf RTL / `project` block design + PS), top, VITIS cpu
`designs/<tasarım>/src, constraints, tb`	HDL, XDC, self-checking testbench'ler (`tb_*.vhd|v|sv`)
`designs/<tasarım>/tcl/bd.tcl`	Block design (project mode)
`designs/<tasarım>/sw/src`, `sw/create_vitis.py`	Firmware kaynakları ve Vitis script'i (işi template'in `scripts/vitis_sw.py`'sine devreder)
`designs/<tasarım>/test.yaml`	Twin ve board testinin beklentileri: `timeout_s`, `expect`, `fail_on`, `summary`
`boards/<kart>.yaml`	Kart: vendor, tool, tool_version, part, pinler, clock, lab programlayıcı; isteğe bağlı `twin:`
`scripts/flow.mjs`, `flow.tcl`, `common/`	Akış; template'e aittir, `factory template update` ile güncellenir
`out/<tasarım>/`	Çıktı (gitignore): `artifacts/` altında summary.json, .bit, .xsa, .elf, raporlar, twin ve board sonuçları
`requirements/`, `handoffs/`, `runs/`	Requirement dosyaları, handoff paketleri, gate kayıtları
`coding-standard/`	Seçilen coding standard, kural listesi ve waiver'lar (§9)

Template ile iki örnek tasarım gelir: `led_counter` (non_project, saf RTL, LED sayacı) ve `hello_ps` (project mode, Zynq PS, UART'a bir satır basan firmware).

Template güncellemesi. aifactory'de template iyileşince proje bunu kendisi ister: proje klasöründe `pnpm factory -- template update` (sadece neyin değişeceğini görmek için `--dry-run`). Komut önce aifactory'yi `git pull` ile günceller (gerekirse `pnpm install`), sonra projede `template/aifactory-<commit>` branch'ini açıp template'e ait dosyaları yazar, `pnpm check`'i koşturur, push eder ve MR açar; proje `main`'de kalır. Testi MR'ın pipeline'ı yapar, geçince merge edilir. Proje `main`'de değilse, commit edilmemiş değişiklik varsa ya da açık bir template MR'ı varsa hiçbir şeye dokunmadan durur ve nedenini söyler. aifactory proje listesi tutmaz; komut hangi projede çalışıyorsa onu günceller. Tasarımlar (`designs/`), board dosyaları, `.env` ve requirement'lar değişmez.

4. Bir requirement'ın yaşam döngüsü

1. **Aç:** `pnpm factory -- requirement new "<başlık>" --mode handoff`. Terminalde fazlar checkbox ile sorulur; script'te `--phases spec,sim,build` verilir. Seçim requirement dosyasının başına `phases:` olarak yazılır.
2. **Yaz:** requirement dosyasındaki Design / Behaviour / Build / Acceptance Criteria bölümlerini doldurun. RTL değişikliği requirement'ta önerilir, mühendis onaylar.

3. **Gönder ve başlat:** `requirement submit RQ-n` handoff paketini üretir; `handoff-begin <run>` temiz çalışma ağacında run'ı açar.
4. **Geliştir:** hızlı döngü için tek tek stage koşturun (bkz. §5). Handoff paketinde kurallar, RAG bağlamı ve kısıtlar vardır.
5. **Gate'leri koştur:** `handoff-finish <run>` requirement'ın fazlarını sırayla koşar; ilk düşen fazda durur, sonrakileri "skipped" olarak işaretler. Daha azını koşmak için `--phases sim` (run açık kalır).
6. **Kaydet:** kod commit'i (`feat(RQ-n): …`) ve run artifact'leri commit'i (`chore(RQ-n): capture handoff-finish gate artifacts`), sonra `approve <run> --by "<ad>"` ve push.
7. **CI:** merge request pipeline'ı requirement'ın fazlarını koşar; main ve tag'de tüm fazlar koşar.
8. **Kapat:** `requirement complete RQ-n --run <run>` MR'ı merge eder; tamamlama commit'i yeni bir pipeline başlatır, komut "checks pending" derse bitince tekrar çalıştırın.

> Geçmiş bir run sonradan değişen kodla yeniden kullanılamaz: kod değiştiyse yeni run açın (`handoff RQ-n` → `handoff-begin` → `handoff-finish`).

5. Akışı elle koşturmak: `pnpm flow`

KOMUT	NE YAPAR	NEREDE
`pnpm flow check`	designCheck: board/proje/bd tutarlılığı, mutlak yol taraması	yerel
`pnpm flow sim [tasarım]`	xvlog/xvhdl/xelab/xsim ile testbench'ler	Vivado host
`pnpm flow synth | impl | bit | xsa`	Tek tek build adımları	Vivado host
`pnpm flow build`	synth+impl+bit (+xsa, project mode)	Vivado host
`pnpm flow sw`	Vitis workspace + uygulama build'i, ELF	Vivado host
`pnpm flow all`	check → sim → build → sw	
`pnpm twin [tasarım]`	ELF'i twin image'ında koşturur, test.yaml'a bakar	Docker host (lab)
`pnpm board [tasarım]`	Kartı açar, programlar, konsolu dinler, kapatır	lab host + kart
`pnpm flow clean`	out/ temizliği	

Vivado host `.env`'de `VIVADO_REMOTE_*` ile tanımlıdır (bugün 192.168.1.3, Windows, Vivado/Vitis 2025.2). Proje kaynakları her koşuda host'a gider: `VIVADO_ACCESS=bot` ile sadece host'taki kopyada olmayan veya farklı olan dosyalar (SHA-256 ile karşılaştırılır), projeden silinen dosyalar host'tan da silinir; `ssh` ile her seferinde tüm kaynaklar tar ile. `out/` host'ta kalır ve sadece artifact'ler geri gelir. Bot yolunda hem giden kaynaklar hem dönen sonuçlar parça parça taşınır: parça boyutu `BOT_API_CHUNK_KB` (boş: 8192, yani 8 MB; zayıf bağlantıda örneğin 2048), kopan parça tek başına yeniden denenir. Builder'daki her adım (kaynakların yerleştirilmesi, Vivado/Vitis, sonuçların paketlenmesi) arka planda koşar ve birkaç saniyede bir takip edilir; bot'un adım başına 120 saniyelik sınırı proje büyüklüğünü kısıtlamaz. Host'taki proje adı kişi başınadır

(`<proje>-<BOT_API_AGENT>`), CI kendi adını kullanır; iki kişi aynı Vivado projesini paylaşmaz. Build yaklaşık 8–10 dakika, sim 20 saniye, Vitis 1 dakika sürer.

Önce `out/<tasarım>/artifacts/summary.json`'a bakın: WNS/WHS, LUT/FF/BRAM/DSP, DRC, süreler, build host ve git sha. Raporlar yanındadır.

6. Twin

Twin projenin içinde değildir; ayrı bir projeden release edilen bir Docker image'ıdır (bfi-sw'nin bfi-simulator'ü kullanması gibi). Hangi twin olduğu `.env`'de yazar: `TWIN_GITLAB_URL`, `TWIN_PROJECT`, `TWIN_REGISTRY`, `TWIN_CONSOLE`; boş alanları `boards/<kart>.yaml`'daki `twin:` bölümü tamamlar. Her koşu twin'in son release'ini alır (`TWIN_TAG` sabitler) ve hangi sürümle koştuğunu `artifacts/twin/result.json`'a yazar. `TWIN_PROJECT` boşsa twin fazı "no twin yet" diyerek atlanır.

`test.yaml` twin ve board için ortaktır: `expect` konsolda görünmesi gereken satırlar, `fail_on` görünmemesi gerekenler, `summary` twin'in JSON özetindeki alanlar için min/max.

7. Board

`pnpm board` sırasıyla: lab bot'tan kartın prizini açar (`BOARD_POWER_NAME`), hw_server üzerinden xsdb ile `rst -system` → bitstream → ps7_init → ELF → `con`, konsolu (`BOARD_SERIAL_PORT`) `timeout_s` boyunca dinler, sonucu `artifacts/board/result.json` ve konsol kaydına yazar, prizi yalnızca kendisi açtıysa kapatır. Her xsdb hedefi kablo seri numarasıyla (`BOARD_JTAG_CABLE`) süzülür; aynı hw_server'a başka kartlar da bağlıdır. Flash'a hiçbir zaman yazılmaz.

CI'da board job'ı üç durumda koşar: requirement'ın fazlarında `board` varsa, gece zamanlanmış pipeline'da, ya da `embedded_board_on_demand` elle tetiklenince. `resource_group: lab-board` aynı anda tek koşuya izin verir.

8. CI pipeline

JOB	FAZ	NE ZAMAN
`embedded_check`	spec	her MR ve branch
`embedded_sim`	sim	requirement fazlarında varsa; main/tag her zaman
`embedded_build`	build	aynı; artifact: bit, xsa, elf, raporlar
`embedded_twin`	twin	aynı; TWIN_PROJECT yoksa atlar
`embedded_board` / `_on_demand`	board	§7
`embedded_builder_prune`	—	main'e her push'ta (merge sonrası); builder'da branch'i silinmiş CI kopyalarını, sahibi kalmamış Vitis workspace'lerini ve bir günden eski iş klasörlerini siler; kişisel kopyalara (`<proje>-<ad>`) dokunmaz; hata verse de pipeline'ı düşürmez

MR pipeline'ı requirement dosyasındaki fazlara bakar (`--requirement-phases`); main ve tag'de hepsi koşar. Vivado job'ları runner'dan Vivado host'a SSH ile çıkar; anahtarlar ve host ayarları instance düzeyindeki CI değişkenlerinden gelir.

9. Coding standard (yeni, RQ-0052)

Proje bir coding standard'a bağlanabilir: `--standard cstd --groups misra,bytecraft [--profile default|dal-a]` ile yaratılırken, ya da var olan projede `pnpm factory -- standard use cstd --groups misra,bytecraft` ile. Standart Bytecraft'ın 5001-K-CSTD'sinden türetilmiş kural gruplarından oluşur:

GRUP	KURALLAR	İÇERİK
`misra`	107	MISRA C:2025 kurallarına birebir atıflı C kuralları
`bytecraft`	40	Dosya adları, isimlendirme, biçim, standart kütüphane ve bitfield yasağı, ölçüler
`do178c`	9	DO-178C'nin koddan istedikleri (ölü/erişilemeyen kod, kullanılmayan tanım yok)
`arm-m3`	17	Cortex-M3 assembly kuralları

Seçim (standart, sürüm, gruplar, profil) projenin `factory.config.json`'ında `codingStandard` alanında durur; geliştiriciye göre değişmediği için `.env`'de değil, git'tedir. Grupları değiştirmek için `pnpm factory -- standard use cstd --groups …` yeniden çalıştırılır. `coding-standard/` klasöründe seçimden üretilen `rules.json` ve `index.md` (yürürlükteki kurallar; gate ve CI bunları okur) ile `waivers.yaml` (gerekçeli sapmalar) durur. Kuralın tam metni RAG'dedir (`coding-standards` kaynağı; "C-045 ne diyor" diye sorulur). Her requirement taslağına "Coding Standard" bölümü, her handoff paketine kural listesi girer. Kodu kurallara karşı otomatik kontrol eden `codingStandard` gate'i (`pnpm standard`, RQ-0053) build'den sonra koşar: MISRA kurallarını cppcheck, biçim kurallarını kendi lexical kontrolleri yakalar, kalanı raporda “manuel review” olarak listelenir; ihlal ya düzeltilir ya `waivers.yaml`'a gerekçesiyle yazılır. Rapor `out/coding-standard/report.md`.

10. Bilinen durumlar

- **Windows yol sınırı (259):** Vitis'in gcc'si 259 karakteri aşan yolu açamaz; `LongPathsEnabled` da çözmez, çünkü gcc uzun yol desteğini bildirmiyor. Bu yüzden Windows'ta Vitis workspace'i proje kopyasının içinde değil, `C:\ab\.v\<8 haneli kod>` altındadır ve platformun adı sabittir (`platform`); en uzun yol, proje ve tasarım adı ne olursa olsun 194 karakterdir. `C:\ab` kısa ve gerçek bir klasördür (junction değil: Vivado junction'ı uzun hedef yoluna çözer). Eski bir template'le açılmış tasarımın `sw/create_vitis.py`'si workspace'i hâlâ `out/<tasarım>/vitis`'e koyar: akış yolu build'den önce tahmin eder, aşacaksa parçalarıyla durur. Çözüm, o dosyayı template'teki `hello_ps/sw/create_vitis.py` ile değiştirmektir (dosyada tasarım adı geçmez).
- **Dosyalar 200 KB'da kesik geliyorsa:** Windows OpenSSH'ın SFTP'si; akış Windows host'ta `scp -O` kullanır. Bitstream'ler ~4 MB tam gelmeli.
- **`vitis -s` hata verse de 0 ile çıkar:** ELF kanıttır; eski ELF build'den önce silinir, yoksa stage düşer.
- **Vivado sürümü:** board dosyasındaki `tool_version` ile host'taki sürüm eşleşmezse build durur; sürüm değişikliği ayrı requirement'tır (IP'leri yükseltir).
- **Build host'ta aynı anda tek Vivado:** iki build çakışırsa ikincisi bekler; ilk pipeline'da sim nadiren "exit 5" ile sessizce düşebiliyor, yeniden deneyin.
- **Board "no target with cable serial":** kart kapalı ya da kablo başka hw_server'da; bot güç durumunu `power/status` ile verir.

11. Komut özeti

```
pnpm factory -- new <proje> --template embedded --board zedboard [--standard cstd --
groups …]
pnpm factory -- env-check                       # eksik .env değerleri
pnpm factory -- ci-variables --write --with-secrets
pnpm factory -- requirement new "<başlık>" --mode handoff [--phases
spec,sim,build,twin,board]
pnpm factory -- requirement submit RQ-n
pnpm factory -- handoff-begin <run>   ·   pnpm flow <stage>   ·   pnpm factory --
handoff-finish <run> [--phases …]
pnpm factory -- approve <run> --by "<ad>"   ·   git push
pnpm factory -- requirement complete RQ-n --run <run>
pnpm factory -- template update                  # önce aifactory'yi günceller, sonra
projeyi güncel template'e getiren MR'ı açar
pnpm factory -- template update --dry-run        # sadece neyin değişeceğini listeler
pnpm flow prune                                  # builder'daki artıkları elle temizler
(CI zaten yapar)
pnpm factory -- standard use cstd --groups misra,bytecraft
```

Hazırlayan: Batur Orkun, Bytecraft · 20 Ekim 2026 · aifactory embedded template, RQ-0044…RQ-0071 durumu